Training — the collection's shared cells

nb022 · 28 June 2026

Abstract

The collection's single training hub: every cell is trained once to a shared root, and the analysis notebooks load those weights instead of retraining (train-once / reuse-many, see ar016). Status **draft** — nothing is trained to the gamma standard yet.

Methods

1. The compute cost

Each trial is surrogate-gradient backprop-through-time: 2000 timesteps ($T = 200$ ms at $\Delta t = 0.1$ ms) over a 1280-neuron conductance network. What makes it costly:

- **Fine timestep.** COBA synapses inject $g(V - E)$, stiffening the membrane so $\Delta t = 0.1$ ms — 10× finer than a current-based model, and non-optional: the gamma rhythm lives there.
- **Sequential.** The 2000 steps are a dependency chain no hardware parallelises, so the job is **bandwidth-bound** — a GPU beats a CPU only ≈ 10×, not the 50–100× of large matmuls.
- **Memory-heavy.** The backward pass stores every step's activations — ≈ 12 GB at batch 256.
- **Scale.** 87 cells × 50 epochs ≈ **185 A100-GPU-hours** (≈ 49M sample-forwards) — ≈ 159 days on CPU, so every cell trains on a GPU.

2. Gold star trainings

87 cells across five families, each fixing its time constants, timestep, and MNIST fraction (re-split 80/20). The standard is $\tau_{\text{AMPA}} = 2$ ms, $\tau_{\text{GABA}} = 6$ ms (loop in gamma, ≈ 44 Hz); the sweeps each vary one axis — spike budget, τ_GABA, timestep (nb044), or init.

Family	τ_{GABA} (ms)	Δt (ms)	Epochs	MNIST	Cells	At spec
canonical (θ_u off)	6	0.1	50	all (70k)	6	0 / 6
θ_u sweep	6	0.1	50	10%	36	0 / 36
τ_GABA ladder	4.5 – 27	0.1	50	10%	18	0 / 18
Δt sweep	6	0.05 – 1.0	50	10%	15	0 / 15
init variants	6	0.1	50	10%	12	0 / 12
total					**87**	**0 / 87**

The **canonical** family (coba, ping at θ_u = off) is the full-MNIST reference the sweeps are read against. *At spec* means trained to the gamma standard — **none yet**: the cells on disk from the original notebooks are at the old standard (2.9–5% MNIST, $\tau_{\text{GABA}} = 9$ ms), so the run trains all 87 fresh.

Choices behind the table:

- **50 epochs, halved from 100.** Accuracy plateaus by ≈ 15–20 epochs (nb024); 50 keeps a ≈ 30-epoch tail for the post-convergence *dynamics* the collection studies (rate drift, confidence inflation) while nearly halving the run.
- **3 seeds throughout.** Every cell — canonical and every sweep — trains three seeds (42, 43, 44), so each point on each frontier carries a cross-seed band and the headline effects (PING's rate attractor) read as robust, not one-run luck. The θ_u interior used to be single-seed; it no longer is.

- **Full MNIST vs 10%.** The canonical reference carries the headline numbers, so it sees all 70k; the sweeps need only the trend across their parameter, so 10% suffices and cuts their cost tenfold.

3. Compute options

Bandwidth is the main driver but not the whole story — the RTX 4090 below has half the A100′s bandwidth yet measured *faster*. Costs and wall-clocks cover the full 50-epoch registry; measured where the card was to hand, projected for the 5090:

Option	GPU (bandwidth)	Samples/s	Full-run cost	Wall-clock
Modal	A100 (2.0 TB/s)	74	≈ $615	≈ half a day (fans out)
RunPod / Vast.ai	RTX 4090 fleet (1.0 TB/s)	100 (meas.)	≈ $47 – 95	≈ 10 hr on ≈ 15 pods (≈ 5.7 d serial)
Cambridge Wilkes3	A100	74	≈ £102	offline through 2026
benjy (CUED, shared)	A6000 (768 GB/s)	26.5	£0	≈ 22 days
Owned workstation	RTX 5090 (1.8 TB/s)	≈ 150 (proj.)	≈ £4,700 once	≈ 3.8 days

The 4090 (≈ 100 samples/s, measured) beats both its bandwidth projection (≈ 37) and the A100, so the 5090 is scaled from it (≈ 1.5×). Wilkes3 is cheapest-and-fast but offline through 2026; benjy is one shared GPU forcing an older PyTorch.

The compute option we selected

The run fans out across RunPod **RTX 4090s**, split by stakes: the 6 heavy canonical cells (≈ 9.7 hr each) on **secure on-demand** — non-preemptible, so a mid-run death is rare — and the 81 light sweep cells (≈ 1 hr each) on cheaper **community** pods, where a lost cell costs an hour, not a day. Together ≈ $67, against Modal's ≈ $615, and unlike the 5090 the 4090s actually provision. A pre-baked cu128 Docker image lets pods boot ready to train; a launcher buckets the 87 cells so the six canonical cells land on separate pods (pinning wall-clock to the ≈ 9.7 hr floor of one canonical cell), with a done-marker check keeping the run idempotent and resumable. Artifacts sync back to the shared root; the figures below build locally. At ≈ 15 pods (6 secure + 9 community) it finishes in ≈ 10 hours — more do not help, since one canonical cell cannot be split. The owned **5090** is the long-term answer for routine iteration.

4. Saved per cell

Each training writes one directory under the shared root (*src/artifacts/notebooks/training//*), with the same files every time — the contract the analysis notebooks read:

File	**Holds**
weights.pth · weights_final.pth	trained model state (all parameter tensors) — best-accuracy epoch **and** final epoch, which differ once the rate attractor drifts past the accuracy plateau (nb024).
config.json	full run config — model, lr, epochs, dt, t_ms, tau_gaba, θ_u, w_in, ei_strength, ei_ratio, readout_w_out_scale,

File	Holds
	max_samples, n_params, and provenance (git sha, run id, env hash).
metrics.json	summary + per-epoch history (below).
metrics.jsonl	the per-epoch records, one JSON per line (live streaming log).
test_predictions.json	predicted vs true label per test image.
output.log · run.sh	training stdout; the exact re-runnable train command.

Top-level fields in metrics.json: **mode, schema_version, model** (run mode, artifact-schema version, model name); **run_finished_at**; **config** (key fields incl. the sweep axes; full set in config.json); **init, end** (dynamics before/after — rate_e, rate_i, cv, act, contrast, f0_hz); **epochs** (per-epoch records, below); **best_acc, best_epoch**; **total_elapsed_s**; **perf** (device, torch, peak memory, epoch times, samples/sec).

Each per-epoch record — one entry in the *epochs* list, one line in metrics.jsonl:

Field	Holds
ep, acc	epoch number; test accuracy (%).
loss, test_loss	mean train / test cross-entropy loss.
rate_e, rate_i, test_rate_e, test_rate_i	mean E / I firing rate, train and test (Hz).
cv, act, contrast, f0_hz	ISI coefficient of variation; active hidden fraction; E-population rhythmicity (lobe−trough contrast in $[0, 1)$) and gamma frequency (Hz, from the autocorrelogram lobe lag; null when no rhythm resolves).
lobe_lag_ms, trough_lag_ms, iei_mode_lag_ms, lobe_to_trough	raw rhythmicity scalars behind contrast/f0 — autocorrelogram lobe and trough lags, inter-event-interval mode lag, unbounded lobe/trough ratio.
lr, grad_norm	learning rate; clipped global gradient norm (epoch mean).
grad_norms, grad_ratios, weight_norms	per-weight-type gradient norm, update/weight ratio, and Frobenius weight norm — keyed by parameter name: W_ff.0 (input), W_ff.1 (readout), plus W_ei.1, W_ie.1 only in the init family's trainable-loop cells (gnorm___ in the jsonl).
test_margin, test_confidence, test_logit_scale	logit-discrimination diagnostics (nb024) — true−runner-up logit margin, softmax confidence in the answer, mean |logit|.
skipped_steps, new_best, samples	steps skipped on non-finite gradients; new-best flag; samples seen.
elapsed_s, train_compute_s, eval_s, observe_s	cumulative wall time + per-epoch timing breakdown.

Results

1. Canonical reference (θ_u off, all MNIST)
coba and ping at θ_u = off, all 70k images, seeds 42/43/44 (6 cells) — the full-data baseline. — **not trained yet**

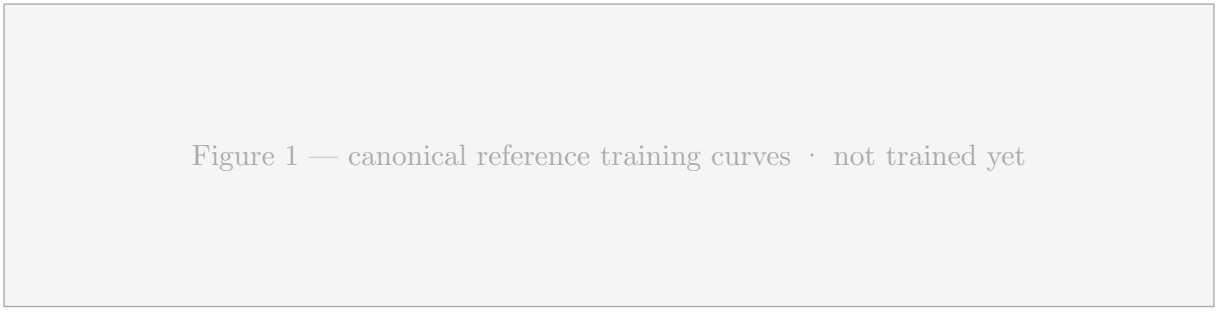

Figure 32: Test accuracy over epochs, canonical full-MNIST cells. **Not trained yet** — new cells awaiting the canonical run.

2. θ_u spike-budget sweep
coba and ping across θ_u ∈ off, 5, 2, 1, 0.5, 0.2, three seeds each (36 cells) — so the accuracy–rate frontier carries error bars at every point. θ_u is the spike budget in spikes/trial. — **not trained yet**

Figure 33: Test accuracy over epochs across the θ_u sweep. Tighter budgets (smaller θ_u) plateau lower — the spike-economy trade-off.

3. τ_GABA ladder
ping across τ_GABA ∈ 4.5, 6, 9, 12, 18, 27 ms, three seeds each (18 cells). — **not trained yet**

Figure 34: Test accuracy over epochs across the τ_GABA ladder; cells converge to similar accuracy regardless of inhibitory decay.

4. Δt sweep
ping across Δt ∈ 0.05, 0.1, 0.25, 0.5, 1.0 ms (physical T fixed), three seeds each (15 cells) — the documented timestep exception. — **not trained yet**

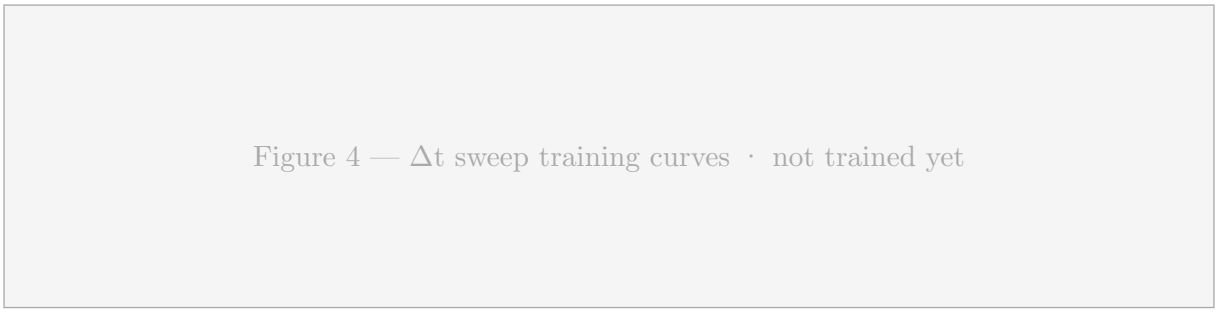

Figure 35: Test accuracy over epochs across the integration-timestep sweep.

5. Init variants

ping with four recurrent-loop inits — frozen PING, trainable from PING / zero / small seed — three seeds each (12 cells). — **not trained yet**

Figure 36: Test accuracy over epochs across the recurrent-loop inits; trainable-loop cells learn noisier curves than the frozen control.